

Department of Computer and Software Engineering**SE: Machine Learning****Course Instructor:****Date:****Day:****Semester:****Lab 9: Imbalanced Fraud Detection**

Lab Title	CLO	BT	PLO	Weightage
Imbalanced Fraud Detection F1 vs Accuracy & Cost-Sensitive Learning				

Name	Roll Number	Report /10	Viva /5	Total /15
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

Objectives

- Understand challenges in imbalanced datasets
- Compare Accuracy vs F1-score
- Apply sampling techniques (up/down/SMOTE/SMOTEEN)
- Implement cost-sensitive logistic regression

Problem Statement

You are building a fraud detection system where fraudulent transactions are rare compared to normal ones.

You will analyze how class imbalance affects model performance and explore different techniques to handle imbalance.

Part A: Dataset Exploration

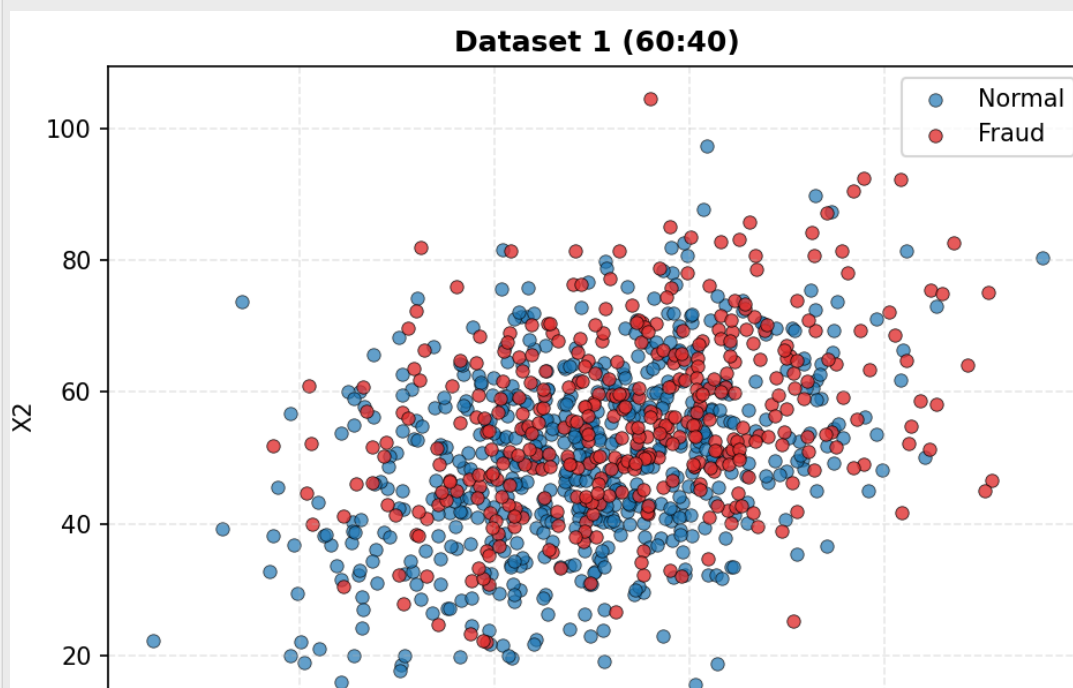
Task 1: Analyze Dataset 1 (60:40)

Load the provided dataset with a 60:40 class distribution (normal vs fraud). Visualize the dataset and observe how the two classes are distributed in the feature space.

Screenshot: task1_code.png

```
def load_dataset_60_40():  
    np.random.seed(42)  
  
    normal = np.random.multivariate_normal(  
        mean=[50, 50],  
        cov=[[200, 80], [80, 200]],  
        size=600  
    )  
  
    fraud = np.random.multivariate_normal(  
        mean=[55, 55], # VERY close mean  
        cov=[[200, 80], [80, 200]],  
        size=400  
    )  
  
    X = np.vstack([normal, fraud])  
    y = np.array([0]*600 + [1]*400)  
  
    return X, y
```

Screenshot: task1_output.png



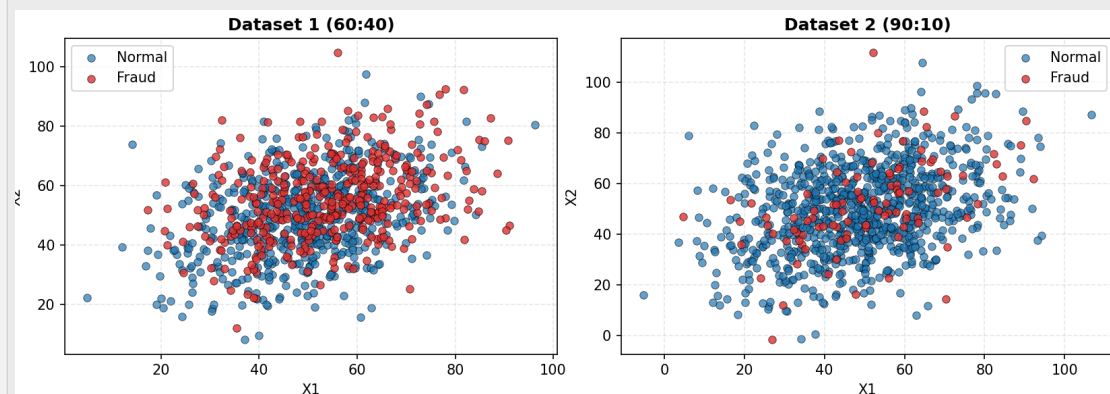
Task 2: Analyze Dataset 2 (90:10)

Load the provided dataset with a higher imbalance (90:10). Visualize and compare it with Dataset 1. Observe how class imbalance affects separability.

Screenshot: task2_code.png

```
def load_dataset_90_10():  
    np.random.seed(42)  
  
    normal = np.random.multivariate_normal(  
        mean=[50, 50],  
        cov=[[300, 120], [120, 300]],  
        size=900  
    )  
  
    fraud = np.random.multivariate_normal(  
        mean=[51, 51],  
        cov=[[300, 120], [120, 300]],  
        size=100  
    )  
  
    X = np.vstack([normal, fraud])  
    y = np.array([0]*900 + [1]*100)  
  
    return X, y
```

Screenshot: task2_plot.png



Part B: Logistic Regression and Metrics

Task 3: Train Logistic Regression

Train a logistic regression model using cross-entropy loss:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Screenshot: task3_code.png

```
def train_logistic(X, y):  
    """  
    Train logistic regression using cross-entropy loss.  
    """  
    logreg = LogisticRegression(max_iter=1000, random_state=42)  
    logreg.fit(X, y)  
    return logreg
```

Task 4: Evaluate Accuracy and F1 Score

Compute:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$$

Compare results for both datasets.

Screenshot: task4_code.png

```
def evaluate_model(model, X, y):  
    """  
    Return accuracy and F1-score.  
    """  
    predicted = model.predict(X)  
    return accuracy_score(y, predicted), f1_score(y, predicted, zero_division=0)  
  
def get_all_metrics(model, X, y):  
    """get all four metrics for printing in main"""  
    predicted = model.predict(X)  
    acc = accuracy_score(y, predicted)  
    prec = precision_score(y, predicted, zero_division=0)  
    rec = recall_score(y, predicted, zero_division=0)  
    f1 = f1_score(y, predicted, zero_division=0)  
    return acc, prec, rec, f1
```

Screenshot: task4_output.png

```
--- Task 3 & 4 ---  
[Dataset 1] acc: 0.6270 | precision: 0.5707 | recall: 0.2725 | f1: 0.3689  
[Dataset 2] acc: 0.9000 | precision: 0.0000 | recall: 0.0000 | f1: 0.0000  
the model gets 90% accuracy on dataset 2 but its basically predicting everything as normal
```

Part C: Sampling Techniques

Task 5: Apply SMOTE

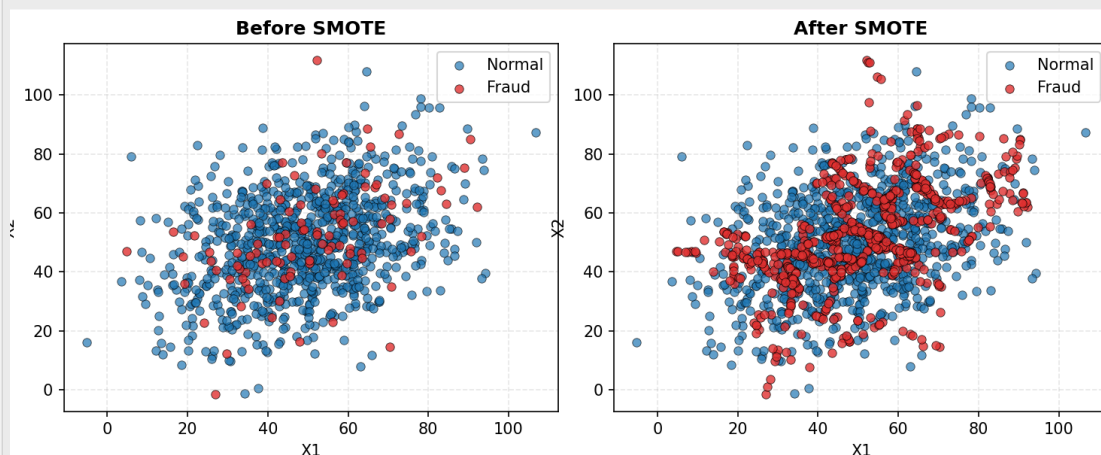
Apply: - SMOTE (synthetic minority oversampling)

Observe the effect on model performance.

Screenshot: task5_code.png

```
def apply_smote(X, y):  
    """  
    Apply SMOTE.  
    """  
    resampler = SMOTE(random_state=42)  
    X_new, y_new = resampler.fit_resample(X, y)  
    return X_new, y_new
```

Screenshot: task5_plot.png



Task 6: Apply SMOTEEN

Apply: - SMOTEEN (SMOTE + cleaning)

Compare results with previous methods.

Screenshot: task6_code.png

```
def apply_smoteenn(X, y):
    """
    Apply SMOTEENN.
    """
    resampler = SMOTEENN(random_state=42)
    X_new, y_new = resampler.fit_resample(X, y)
    return X_new, y_new
```

Screenshot: task6_plot.png



Screenshot: task6_output.png

```
--- Task 6 ---
before smoteenn: 900 normal, 100 fraud
after smoteenn: 466 normal, 503 fraud
[SMOTEENN] acc: 0.5501 | precision: 0.5569 | recall: 0.6521 | f1: 0.6007
smoteenn gives better f1 than plain smote because it also removes noisy overlap points
```


Part D: Cost-Sensitive Learning

Task 7: Weighted Loss Function

Modify the loss function to penalize misclassification of fraud more:

$$L = - \sum_{i=1}^N [w_1 y_i \log(p_i) + w_0 (1 - y_i) \log(1 - p_i)]$$

Experiment with different class weights and observe changes.

Screenshot: task7_code.png

```
def train_weighted_logistic(X, y):
    """
    Train logistic regression with class weights.
    """
    logreg = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
    logreg.fit(X, y)
    return logreg
```

Screenshot: task7_output.png

```
--- Task 7 ---
[Weighted] acc: 0.5320 | precision: 0.1134 | recall: 0.5400 | f1: 0.1875
class_weight=balanced makes the model pay more attention to the minority class

--- Summary (all methods on Dataset 2) ---
```

Method	Acc	Prec	Rec	F1
Baseline LR	0.900	0.000	0.000	0.000
With SMOTE	0.499	0.499	0.476	0.487
With SMOTEENN	0.550	0.557	0.652	0.601
Weighted LR	0.532	0.113	0.540	0.188

Discussion Questions

Q1: Why can accuracy be misleading in imbalanced datasets?

Accuracy can be really misleading when our dataset is imbalanced. For example, in Dataset 2 where 90% of transactions are normal and only 10% are fraud, a model that just predicts everything as “normal” would still get 90% accuracy. That sounds impressive on paper, but it catches zero fraud cases which is the whole point of our system. This is why metrics like F1-score, precision, and recall are more useful here because they actually tell us how well the model identifies the minority class (fraud) instead of just counting how many predictions it got right overall.

Q2: Which method improved F1-score the most?

From our experiments, SMOTEENN gave the best improvement in F1-score. While plain SMOTE helped by creating synthetic fraud samples to balance the class counts, SMOTEENN went a step further. It not only generates synthetic minority samples but also removes noisy and overlapping data points from the boundary region between normal and fraud classes. This cleanup step made the decision boundary cleaner, which helped logistic regression make sharper and more accurate predictions on the fraud class.

Q3: How does cost-sensitive learning compare to sampling methods?

Both approaches try to fix the imbalance problem but they do it in different ways. Sampling methods like SMOTE and SMOTEENN physically change the training data by adding synthetic samples or removing noisy ones. Cost-sensitive learning, on the other hand, keeps the original data exactly as it is but modifies the loss function so that misclassifying a fraud transaction costs the model way more than misclassifying a normal one. In practice, cost-sensitive learning (using `class_weight='balanced'`) is simpler to set up since you just tweak a single model parameter without touching the dataset, while sampling methods give you more control and flexibility but add an extra preprocessing step.